University of Hawai'i at Manoa · Campus Energy Project

Code Walkthrough

Multi-Meter Load Forecasting Pipeline

FY2025 · substations_kw_15_min_fy25.csv · XGBoost · Open-Meteo

STEP 1	STEP 2	STEP 3	STEP 4	STEP 5
Pipeline	Features	Training	Visualize	Forecast

How the Code Is Organized

The forecasting system is split into five Python files. Each file does one specific job, and they run in order — each one saves output files that the next file reads. You never need to run them out of order.

File	Job	Time
`step1_pipeline.py`	Load meter data + fetch weather + reshape into daily profiles	~2 min
`step2_features.py`	Build lag features + train/test split for every meter	~30 sec
`step3_train.py`	Train 4 models per meter + evaluate accuracy	~15 min
`step4_visualize.py`	Generate dashboard plots for every meter	~1 min
`step5_forecast.py`	Forecast any future date for any meter	~30 sec

STEP 1

step1_pipeline.py

Data loading, weather fetching, reshaping

This is the foundation of the whole project. It turns your raw CSV file and live weather data into clean daily load profiles that the model can learn from.

Part A — Loading the meter CSV

Reads your 15-minute substation data, splits it by meter ID, and resamples from 15-minute to hourly by averaging every 4 readings. Meters that are mostly zero (like 5B03646 which is 100% zeros) are automatically skipped because there is nothing meaningful to forecast.

```
df = pd.read_csv('substations_kw_15_min_fy25.csv', parse_dates=['datetime'])
df = df.sort_values(['meter_id', 'datetime'])

# Resample 15-min → hourly (mean of 4 readings keeps units in kW)
hourly = sub.resample('h').mean()

# Skip meters with too many zeros (no useful signal)
if zero_frac > ZERO_THRESHOLD:
    continue  # skips 5B03646 (100% zeros)
```

Part B — Fetching weather from Open-Meteo

Calls the Open-Meteo API (free, no account needed) and downloads a full year of hourly weather for UH Manoa. This only happens once — the response is cached locally so re-runs are instant. The same weather data is shared across all meters since they are all on the same campus.

```
params = {
    'latitude':  21.2969,    # UH Manoa
    'longitude': -157.8171,
    'start_date': '2024-07-01',
    'end_date':   '2025-06-30',
    'hourly': ['temperature_2m', 'shortwave_radiation',
               'relative_humidity_2m', 'wind_speed_10m', ...],
    'temperature_unit': 'fahrenheit',
    'timezone': 'Pacific/Honolulu',
}
responses = om.weather_api('https://archive-api.open-meteo.com/v1/archive',
                           params=params)
```

Part C — Engineering weather features

Converts raw weather variables into features that are more directly related to electricity demand. The two most important ones are:

> Cooling Degree Hours (CDH) — measures how hard the AC has to work. Any temperature above 65 F counts toward CDH. A hot day means high CDH means high campus load.

Heat Index — combines temperature and humidity into how hot it actually feels. High humidity at 85 F makes the AC work harder than dry air at 85 F.

```
# Cooling degree hours — direct AC demand proxy
weather['cdh'] = (weather['temp_f'] - 65).clip(lower=0)

# Heat index — how hot it actually feels (Rothfusz formula)
weather['heat_index_f'] = (-42.379 + 2.049*T + 10.143*RH
                           - 0.225*T*RH - 0.007*T**2 ...)

# Cyclical hour encoding — so model knows 11pm and midnight are adjacent
weather['hour_sin'] = np.sin(2 * np.pi * hour / 24)
weather['hour_cos'] = np.cos(2 * np.pi * hour / 24)
```

Part D — Reshaping into daily profiles

Pivots the hourly data so each row is one day and each column is one hour (h00 through h23). This gives you a matrix where every row is the complete 24-hour load curve for one day — which is what the model predicts.

```
# Pivot: rows = days, columns = hours h00..h23
profiles = df.pivot(index='date', columns='hour', values='kw')
           .rename(columns=lambda h: f'h{h:02d}')

# Result shape: [364 days x 24 hours]
# Each row = one day's complete load curve in kW
```

OUTPUT FILES

meter_data/<meter_id>_profiles.csv — daily load profile matrix | meter_data/<meter_id>_features.csv — weather summaries per day | meter_data/active_meters.csv — list of meters to model

STEP 2

step2_features.py

Feature engineering + train/test split

Takes the daily profiles from Step 1 and builds the full set of 131 input features the model trains on. Raw weather alone is not enough — the model also needs to know what load looked like recently.

Part A — Lag profile features

The most important features in the whole model. A lag feature is simply a past day's load curve used as an input. The model learns things like 'when yesterday's 2pm load was high, today's 2pm load is probably also high.'

Feature group	What it is	Why it helps
`Lag-1 (24 features)`	Yesterday's full 24-hour load curve	Strongest predictor — campus load has daily inertia
`Lag-7 (24 features)`	Same weekday last week's curve	Weekly schedule repeats — Monday resembles last Monday
`Lag-2 (24 features)`	Two days ago's curve	Secondary reference for multi-day weather trends
`Roll-7 (24 features)`	7-day rolling average curve	Captures typical load this point in the semester

```
# Lag-1: yesterday's profile — most important feature group
lag1 = profiles.shift(1)                    # shift all rows down by 1 day
lag1.columns = [f'lag1_{h}' for h in HOURS] # rename columns

# Lag-7: same weekday last week
lag7 = profiles.shift(7)

# 7-day rolling mean — excludes today with shift(1)
roll7 = profiles.shift(1).rolling(window=7, min_periods=3).mean()
```

Part B — Temporal train/test split

The last 60 days of data are held out and the model never sees them during training. This is always done by date order — never randomly. Random splitting would let the model accidentally learn from future data, making results look artificially good.

```
# Always split by TIME, never randomly
split = len(X) - 60          # last 60 days = test set
X_train = X.iloc[:split]     # everything before the cutoff
X_test  = X.iloc[split:]     # the final 60 days
Y_train = Y.iloc[:split]
Y_test  = Y.iloc[split:]
```

OUTPUT FILES

meter_data/<meter_id>_X_train/test.csv — feature matrices | meter_data/<meter_id>_Y_train/test.csv — target profile matrices

STEP 3

step3_train.py

Model training and evaluation

Trains four models for every active meter and prints a comparison table showing which one performs best on the held-out test days. The same four models are trained for every meter so you can see which approach wins per meter.

The four models — in order of complexity

Model	How it works	Expected MAE
Persistence	Predict tomorrow = yesterday. No training needed. Copies lag1 features directly.	~400–600 kW
Mean Profile	Compute average curve per day-of-week. Predict Monday avg on Mondays, etc.	~500–900 kW
Ridge Regression	Linear equation across all 131 features. Regularized to prevent overfitting.	~200–400 kW
XGBoost	24 gradient-boosted tree models (one per hour). Main model. Most accurate.	~100–300 kW

XGBoost training — the key section

XGBoost trains 24 separate models — one for each hour of the day. Midnight load and 2pm load are driven by completely different factors, so separate models let each hour learn different feature weights. Early stopping prevents overfitting by halting training when accuracy stops improving on a small validation set.

```
for hour in HOURS:    # loops h00, h01, ... h23
    model = xgb.XGBRegressor(
        n_estimators     = 500,   # max trees (early stopping cuts this)
        max_depth        = 4,     # tree depth — deeper = more complex
        learning_rate    = 0.05,  # step size per tree
        subsample        = 0.8,   # each tree sees 80% of training days
        colsample_bytree = 0.7,   # each tree sees 70% of features
        min_child_weight = 5,     # leaf must have 5+ days — no memorizing
        early_stopping_rounds = 30,  # stop if no improvement for 30 trees
    )
    model.fit(X_train, Y_train[hour],
              eval_set=[(X_val, Y_val[hour])],  # validation for early stop
              verbose=False)
    models[hour] = model
```

Evaluation metrics printed per meter

After training, every model is evaluated on the 60 test days it never saw during training. The same five metrics are computed for all four models so comparison is fair.

Metric	What it measures	Your target
MAE	Average error in kW — most interpretable number	< 400 kW

RMSE	Like MAE but penalizes large errors more heavily	< 600 kW
MAPE	Error as a percentage of actual load	< 5%
Peak Error	How close your predicted daily peak is to actual peak	< 500 kW
Profile r	Pearson correlation between predicted and actual curve shape	> 0.85

OUTPUT FILES

models/<meter_id>_xgb.pkl — saved XGBoost models | meter_data/<meter_id>_pred_*.csv — test predictions | meter_data/model_comparison_all.csv — full metrics table

STEP 4

step4_visualize.py

Dashboard generation for every meter

Reads the predictions saved by Step 3 and generates visual outputs. No new training happens here — it is purely about turning numbers into charts you can read and present.

Per-meter dashboard — 6 panels

Panel	Plot	What to look for
1	Full-year heatmap	Dark band = daytime peak. Light rows = breaks/holidays.
2	Avg profile by day-of-week	Weekend lines should be visibly lower than weekday lines.
3	Predicted vs actual (6 days)	Teal dashed line should closely follow the orange actual line.
4	Hourly MAE comparison	XGBoost line should sit lowest. Peak hours will have highest error.
5	Signed error heatmap	Gray = good. Red = overestimate. Blue = underestimate.
6	Metrics summary cards	Your headline accuracy numbers: MAE, MAPE, profile correlation.

Cross-meter comparison chart

Generates one additional chart that overlays the average weekday and weekend profiles for all five meters on the same axes. This reveals which substations serve different parts of campus — a substation that peaks in the morning serves different buildings than one that peaks in the afternoon.

```
for meter_id in meter_ids:
    profiles = pd.read_csv(f'meter_data/{meter_id}_profiles.csv')
    weekday_mask = profiles.index.dayofweek < 5
    avg_profile  = profiles[weekday_mask][HOURS].mean()
    ax.plot(range(24), avg_profile.values, label=meter_id)
```

OUTPUT FILES

plots/<meter_id>_dashboard.png — per-meter diagnostic dashboard | plots/meter_comparison.png — all meters on one chart

STEP 5

step5_forecast.py

Live forecast for any meter on any future date

This is the real-world use case — predicting load for a future day before it happens. It loads the trained models from Step 3, fetches live weather from the Open-Meteo forecast API, and generates a predicted 24-hour load profile.

How to run it

```
# Forecast tomorrow for ALL active meters
python step5_forecast.py

# Forecast a specific meter only
python step5_forecast.py --meter 5C03646

# Forecast a specific meter on a specific date
python step5_forecast.py --meter 5C03646 --date 2025-06-15

# Forecast ALL meters on a specific date
python step5_forecast.py --date 2025-06-15
```

Part A — Fetch live forecast weather

Calls the Open-Meteo 7-day forecast API for UH Manoa and pulls the 24 hourly weather values for your chosen date. This is where the project crosses from historical analysis into real prediction — the model now uses projected weather instead of measured weather.

```
params = {
    'latitude':  21.2969,
    'longitude': -157.8171,
    'forecast_days': 7,
    'hourly': ['temperature_2m', 'shortwave_radiation', ...],
    'timezone': 'Pacific/Honolulu',
}
# Filter to just the target date's 24 hours
day_data = df[df.index.date == pd.to_datetime(target_date).date()]
```

Part B — Build the feature row

Assembles all 131 features for the forecast day. Weather features come from the live forecast. Lag features come from your actual historical profiles — yesterday's real meter reading, same weekday last week's real reading, and so on.

```
# Lag features: pull from real historical meter data
lag1 = available.iloc[-1]          # yesterday's actual profile
lag7 = lag7_candidates.iloc[-1]    # same weekday last week

for h in HOURS:
    row[f'lag1_{h}'] = float(lag1[h])   # 24 lag-1 features
    row[f'lag7_{h}'] = float(lag7[h])   # 24 lag-7 features

# Align to exactly the columns the trained model expects
X = X[X_train.columns]
```

Part C — Run the 24 models and assemble the forecast

Passes the feature row through each of the 24 hour-models and collects one kW prediction per hour. Assembles them into the final 24-hour load curve and saves a forecast plot.

```
# Each hour-model predicts one kW value
preds = {h: float(xgb_models[h].predict(X.values)[0]) for h in HOURS}

# preds is now a dict: {'h00': 9840.0, 'h01': 9620.0, ..., 'h23': 10100.0}
result = pd.Series(preds)

peak_hour = HLABELS[result.argmax()]  # e.g. '13:00'
peak_kw   = result.max()              # e.g. 14200 kW
```

OUTPUT FILES
plots/<meter_id>_forecast_<date>.png — predicted load profile + weather panel

---

Quick Reference — All Output Files

File / Folder	Created by	Contains
meter_data/<id>_profiles.csv	Step 1	[364 x 24] daily kW profiles per meter
meter_data/<id>_features.csv	Step 1	Weather + calendar summaries per day
meter_data/active_meters.csv	Step 1	List of meters that will be modelled
meter_data/<id>_X_train/test.csv	Step 2	131-feature input matrices
meter_data/<id>_Y_train/test.csv	Step 2	24-hour target profile matrices
models/<id>_xgb.pkl	Step 3	Trained XGBoost models (24 per meter)
meter_data/<id>_pred_xgb.csv	Step 3	Test-day predictions from XGBoost
meter_data/model_comparison_all.csv	Step 3	MAE, MAPE, profile r for all models
plots/<id>_dashboard.png	Step 4	6-panel diagnostic dashboard per meter

`plots/meter_comparison.png`	Step 4	All meters overlaid on one chart
`plots/<id>_forecast_<date>.png`	Step 5	Future load profile forecast with weather